Virtual Cosmos – Assignment

Overview

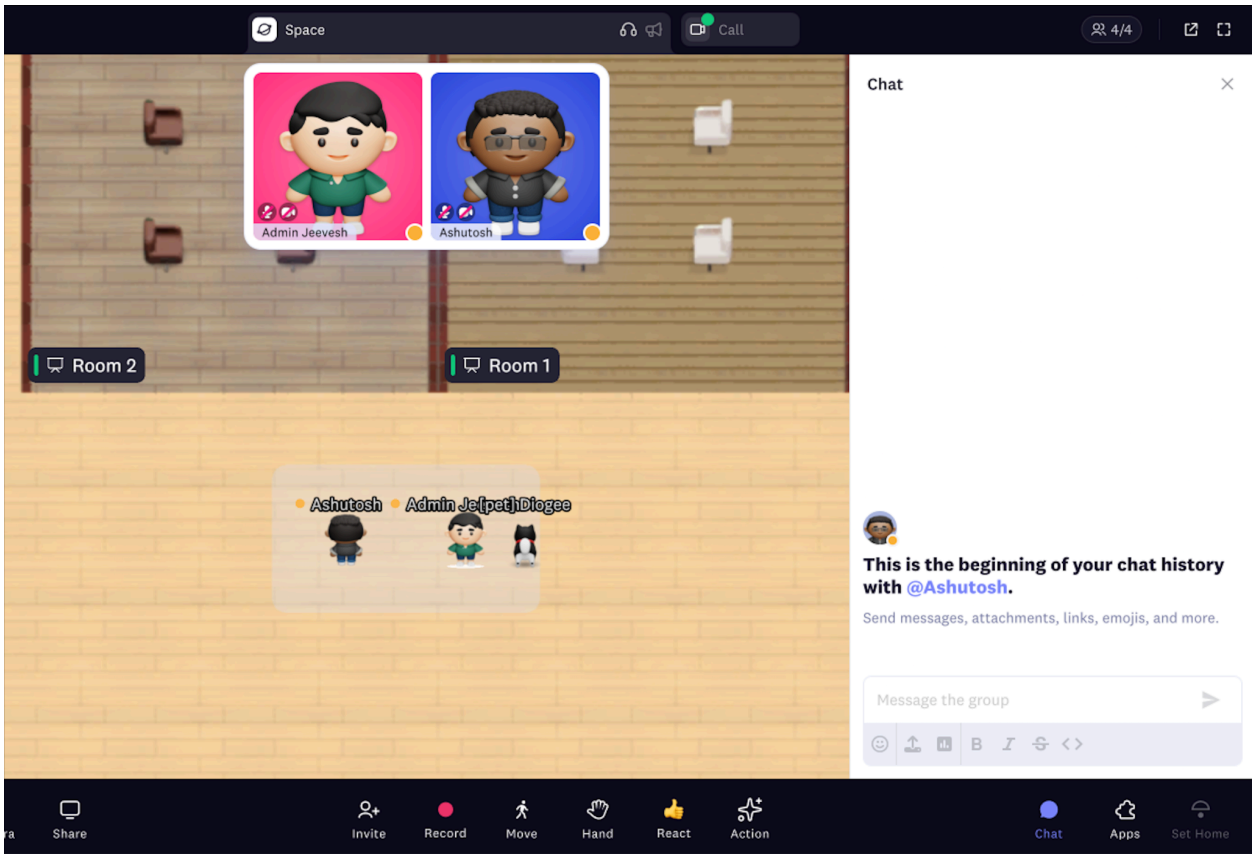
Build a 2D virtual environment (“Cosmos”) where users can move around and interact with each other in real time.

The core idea:

- When users come close → chat connects
- When users move away → chat disconnects

This simulates real-world proximity-based interaction in a virtual space.

👉 **Reference Demo:**
https://cosmos.video/v/5dvm-syhq-p15w/office



Objective

This assignment evaluates:

- Frontend skills (movement, UI/UX, rendering)
- Real-time logic (proximity detection)
- Backend fundamentals (user state, sockets)
- System design thinking

Recommended Tech Stack

Frontend

- React (with Vite)
- PixiJS (for canvas rendering)
- Tailwind CSS

Backend

- Node.js (Express)
- Socket.IO (real-time communication)
- MongoDB (user/session storage)

You are free to use alternatives, but you must justify your choices.

Core Features (Must Have)

1. User Movement

- Render a 2D space using PixiJS
- Represent each user as a circle or avatar
- Enable movement via keyboard (WASD or Arrow keys)

---

2. Real-Time Multiplayer

- Multiple users should be visible on the screen
- Sync user positions in real time using sockets

---

3. Proximity Detection (Core Logic)

- Define a radius zone around each user
- If the distance between two users < radius → connect
- If distance ≥ radius → disconnect

---

4. Chat System

- When users are within proximity:
  - Automatically join the same chat room
  - Enable messaging UI
- When users move apart:
  - Automatically leave the chat room
  - Disable messaging UI

---

5. UI/UX (Important)

- Clean and minimal interface
- Display:
  - User avatars
  - Active connections
  - Chat panel (only when connected)

Backend Requirements

Track the following:

- `userId`
- `position (x, y)`

- Active connections

Use Socket.IO for:

- Real-time position updates
- Connection/disconnection events
- Chat messaging

User Flow (Experience)

1. User opens the app and enters the cosmos
2. Sees other users moving in real time
3. Moves closer to another user
4. Chat panel appears → connection established
5. Moves away
6. Chat panel disappears → connection terminated

Submission Guidelines

1. GitHub Repository

- Well-structured codebase
- Clear README
- Setup and run instructions

---

2. Demo Video (2–5 minutes)

Demonstrate:

- User movement
- Real-time interaction
- Chat connect/disconnect behavior
- Any bonus features (if implemented)

Submission Guidelines

Submit the following:

1. GitHub Repository

- Proper README
- Setup instructions
- Clean code

2. Demo Video (2–5 mins)

Show:

- Movement
- User interaction
- Chat connect/disconnect
- Any bonus features

Submit here: https://forms.gle/GtkmYbjw4FVkrCzB8